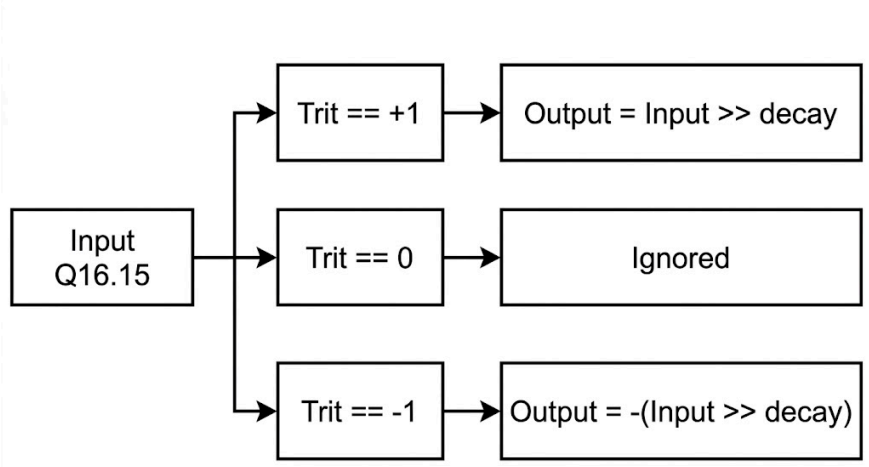
Quantized Adaptive Trainer Neural Network - Hybrid Fixed Point

1. Architettura di Base e Forward Pass

Questo documento presenta **QATNN** (Quantized Adaptive Trainer Neural Network), un'architettura basata sul **paradigma del Quantization-Aware Training** (QAT), distinguendosi tuttavia per l'adozione di un **regime ternario governato da pesi ombra** (Shadow Weights) e una **mappa di isteresi dinamica**.

- **Pesi Ternari**: Durante l'inferenza, i pesi assumono solo valori {-1, 0, +1}.
- **Inferenza e Propagazione Vettoriale**: Come proprietà dei pesi ternari possiamo quindi sostituire i pesanti calcoli matriciali con semplici somme o sottrazioni logiche.



***Nota di implementazione**:* Il parametro **decay** è specifico per ogni layer (o canale) ed è salvato come intero costante derivato in fase di training.

Fattore di Scala Globale (γ_{shift}):

Per ripristinare il magnitudo del segnale perso durante l'appiattimento della quantizzazione ternaria, il sistema utilizza un fattore di scala γ_{raw} specifico per ogni layer (o canale) L'accumulo dei valori assoluti dei pesi ombra viene eseguito tramite un **registro temporaneo a 64 bit** (`int64_t sum_abs = 0;`) per prevenire l'**overflow aritmetico** prima dell'applicazione del bit-shift.

- **Giustificazione architetturale:** I test empirici dimostrano che il fattore di scala è la componente strutturale portante della quantizzazione: omettere questa compensazione (forzando $\gamma = 1$) compromette gravemente l'ottimizzazione del modello.
- **Calcolo del Fattore di Scala (γ_{raw} e γ_{shift}):** Per eliminare l'uso di funzioni in virgola mobile (**double**) e garantire la compatibilità nativa con l'aritmetica intera della ALU, il calcolo sfrutta istruzioni hardware a bit (**__builtin_clz**) al posto di **log2**:

$$\gamma_{raw} = (\Sigma|Shadow_i|) \gg (31 - __builtin_clz(N))$$

- Dove N è la **dimensione fissa del blocco**, rigorosamente pari a una potenza di due (es. 4096), garantendo la corrispondenza esatta del logaritmo intero a bit.
- Il fattore di **shift globale** viene quindi ricavato come:

$$\gamma_{shift} = 31 - ((\gamma_{raw} == 0)?\, 31 : __builtin_clz(\gamma_{raw})) - 15$$

- **Applicazione finale (Shift Puro):** Il layer conclude l'accumulo senza moltiplicazioni, scalando l'output grezzo:

$$\text{Se } \gamma_{shift} > 0 \Rightarrow Output_{shiftato} = Output_{grezzo} \ll \gamma_{shift}$$

$$\text{Se } \gamma_{shift} < 0 \Rightarrow Output_{shiftato} = Output_{grezzo} \gg |\gamma_{shift}|$$

SiLUNS (SiLU Negative Shift): Funzione di Attivazione a Divisione Zero per Motori QATNN in Q16.15

Per rispondere ai **vincoli architetturali del motore neurale a virgola fissa Q16.15**—specificatamente il **divieto di impiegare aritmetica in virgola mobile**, calcoli esponenziali onerosi e circuiti di divisione hardware—introduciamo **SiLUNS** (*SiLU Negative Shift*).

SiLUNS **approssima le proprietà matematiche** della funzione di attivazione **SiLU** standard ($f(x) = x \cdot \sigma(x)$), **mappando** interamente il **comportamento su operazioni di bit-shift** e confini lineari a tratti.

1. Formulazione Matematica e Vincoli Strutturali

Operando all'interno del formato dati **Q16.15**, il **campo** dinamico di **sicurezza** previsto per **SiLUNS è l'intervallo [-2.0, +1.99997]**; sebbene il formato saturi nativamente a $\pm\ 65535$, il ramo di risalita richiede questo limite per **non attraversare lo zero verso un'uscita errata** (positiva). La mappa operativa è **divisa attraverso due rami principal**i:

- **Passaggio Positivo ($x \geq 0$):**

$$f(x) = x$$

  I segnali mantengono la totale fedeltà, evitando attenuazioni per attivazioni fortemente positive.

- **Regione Negativa a "Sacca" ($0 > x \geq X_{min}$):**

  Per replicare la caratteristica **gobba negativa della SiLU approssimando un interno curvo** e garantire una continuità assoluta (C^0) priva di salti che distruggerebbero la retropropagazione, il **dominio negativo è diviso nel punto di minimo teorico** ($X_{min} = -\ 41900$, corrispondente a $\approx -\ 1.28$). Il raccordo avviene tramite due rami a pendenza invertita compensati da un offset algebrico costante:

  - **Ramo di Discesa ($0 > x > -\ 41900$):**

$$f(x) = x \gg 2$$

  - Il segnale viene attenuato dolcemente fino a raggiungere il **minimo relativo esatto** nel punto di flesso: $f(-\ 41900) = -\ 10475$ in **Q16.15** (circa $-\ 0.319$, un'approssimazione del vero minimo della SiLU a $-\ 0.278$).

- **Ramo di Risalita Raccordato ($x \leq -41900$):**
  Oltre il minimo, la funzione deve **risalire verso lo zero per smorzare gli *outliers* negativi**. Invece di azzerare brutalmente il segnale (causando un salto di discontinuità), si **inverte la pendenza bitwise e si applica un offset** per raccordare perfettamente il valore a quello del ramo precedente:

$$f(x) = -(x \gg 2) - 20950$$

- ***Dimostrazione di continuità**:*
  Al confine $x = -41900$, l'equazione restituisce $-(-10475) - 20950 = 10475 - 20950 = -10475$, **unendosi al ramo di discesa senza alcun salto**. Man mano che x diventa più negativo, il termine $-(x \gg 2)$ diventa positivo, facendo **risalire morbidamente la funzione verso lo zero** (es. a $x = -65536$ la funzione vale -4566).

2. Efficienza Hardware e Impatto sulle Risorse

Sostituendo le **funzioni esponenziali** trascendenti e i **circuiti di divisione** con bit-shift a livello di bit e diramazioni condizionali, **SiLUNS** garantisce l'**esecuzione in tempo costante** ($O(1)$ cicli di clock sulla ALU), **preservando** al contempo **il flusso dei gradienti durante la retropropagazione** ed eliminando qualsiasi overhead di memoria.

Benchmark e Profilazione Prestazionale: I test di validazione sull'**inferenza vettoriale SIMD** dimostrano che la **sostituzione** integrale delle **istruzioni MAC** (Multiply-Accumulate) **con addizioni logiche condizionali** (pesi ternari) e operazioni di bit-shift (SiLUNS e Gamma scaling) genera un **abbattimento drastico del costo computazionale**. Tale accelerazione è il risultato combinato dell'elusione totale dei moltiplicatori fisici sulla ALU e del **packing spinto a 1.625 bit per peso, che riduce dell'80%** (contro 8 bit) **o del 90%** (contro 16 bit) le latenze legate al **memory bandwidth**.

2. Mappa di Isteresi per il ricalcolo dei Trit (Shadow Weight to Trit Mapping)

La conversione dei pesi ombra (Shadow in **Q16.15**) in pesi ternari (**trit**) utilizza una logica di **isteresi** per **prevenire oscillazioni** frequenti:

- Se Shadow >= T:
  - Trit precedente = 1 -> Nuovo Trit = 1
  - Trit precedente = 0 -> Nuovo Trit = 1
  - Trit precedente = -1 -> Nuovo Trit = 0
- Se Shadow <= -T:
  - Trit precedente = 1 -> Nuovo Trit = 0
  - Trit precedente = 0 -> Nuovo Trit = -1
  - Trit precedente = -1 -> Nuovo Trit = -1
- Se -T < Shadow < T:
  - Trit rimane invariato dallo stato precedente

Nota di Inizializzazione: All'avvio del modello, i pesi ombra (**Shadow**) vengono inizializzati con una **distribuzione gaussiana** controllata centrata sullo zero. A differenza delle fasi successive dove **interviene la logica di isteresi**, i pesi ternari (**trit**) allo stato zero **non possiedono uno "stato precedente"** su cui basare il calcolo; pertanto, **vengono anch'essi allocati inizialmente con valori casuali** uniformemente distribuiti nell'insieme $\{-1, 0, +1\}$. Questo approccio garantisce una **distribuzione statistica nativamente bilanciata**, rompendo la simmetria del layer e fornendo la varianza necessaria prima dell'avvio della fase di apprendimento, lasciando poi che la rete corregga i valori iterativamente.

Nota di Stabilità: La **soglia di isteresi** (**T**, definita da una costante ottimizzata,213), opera **esclusivamente come discriminante logico** per la transizione di stato dei trit, consentendo ai pesi ombra (**Shadow**) di evolvere **liberamente nello spazio** continuo del formato **Q16.15** durante le normali fasi di discesa del gradiente.
Per preservare l'integrità numerica dei registri a 32 bit ed **evitare saturazioni distruttive** senza bloccare la dinamica di apprendimento, la **compensazione algebrica** (mediante sottrazione o addizione del relativo offset di correzione) viene attivata **esclusivamente in prossimità del limite strutturale** critico del registro ospitante (**`INT32_MAX`**), che funge da contenitore fisico primario per **prevenire il wrap-around**, ben oltre i normali margini di saturazione del formato matematico **Q16.15**. Tale meccanismo **garantisce che il peso rimanga continuo** e privo di vincoli artificiali nella sua fascia operativa ordinaria, intervenendo unicamente al **raggiungimento dei tetti estremi** per preservare i resti in virgola fissa e **prevenire l'overflow hardware**.

Gestione del Trit a Zero (Forward vs Backpropagation):

- **In Inferenza (Forward):** Il salto condizionale sui trit nulli **viene omesso** per non interrompere la vettorizzazione (SIMD) della CPU. L'elaborazione **procede per blocchi continui tramite operazioni standard di moltiplicazione e addizione**, calcolando regolarmente anche i valori a zero **per mantenere la pipeline hardware satura e allinearsi all'efficienza dei kernel vettoriali**.
- **In Backpropagation (STE e Sblocco Isteresi):** I trit uguali a 0 non devono **azzerare il gradiente dello Shadow Weight**. Applicando lo **Straight-Through Estimator** (STE), il passaggio viene **trattato come attivo (valore di default)**, evitando che lo Shadow Weight rimanga intrappolato nello zero e permettendogli di **accumulare variazioni per uscire dalla isteresi**.

3. Struttura Fisica dei File e Gestione Memoria (Data Locality Chunking)

Per permettere il caricamento **dinamico e isolato** basato sulla **percentuale operativa** (%X), i dati sono rigidamente **partizionati in "Chunk"** numerati, dove ogni file rappresenta un layer (o **gruppo di layer**). Il sistema carica in RAM **solo i chunk strettamente necessari**.

File di Inferenza (Serie I_X.bin)

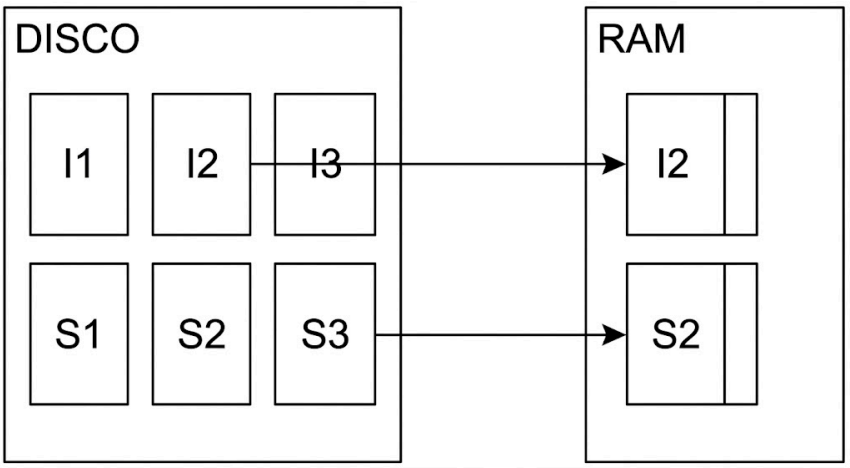
Contengono i dati per il Forward Pass e gli input/output necessari alla Backpropagation per quello specifico blocco.

- **Nomenclatura**: I1.bin, I2.bin, I3.bin, ecc.
- **Struttura interna (contigua)**: [Input] [Trit] [Output] [decay] [gamma_shift].
- **Impacchettamento dei Pesi Ternari (Trit_Packed):** I valori {-1, 0, 1} sono compressi raggruppando **128 pesi in blocchi da 26 byte**. Questo produce una **densità effettiva in RAM di 1.625 bit per parametro** (lasciando un margine di 2 trit inutilizzati per blocco **per favorire l'allineamento in memoria**). A 4096 per 4096 il campo dei trit passa da **16.00 MiB a circa 3.25 MiB**, riducendo l'ingombro di quasi l'**80%**.
- **La compatibilità con il formato adattivo BITCOS** (1.485 bit per peso) si attiva in modo **dinamico all'aumentare della frazione di zeri** appresa dai pesi ternari durante l'addestramento (mirando a una **sparsità strutturale** $\geq 51\%$, in linea con i recenti standard della letteratura sui modelli ternari).
- **Caricamento**: Se si elabora il layer 2, si esegue il mmap unicamente di I2.bin.

File Ottimizzatore (Serie S_X.bin)

Contengono gli accumulatori ad alta precisione usati **solo per la Backpropagation**, con corrispondenza 1:1 rispetto ai file di inferenza.

- **Nomenclatura**: S1.bin, S2.bin, S3.bin, ecc.
- **Struttura interna** (contigua): [Shadow Weights][decay Weight].
- **Caricamento Parallelo:** Durante la backprop del layer 2, il sistema apre I2.bin e S2.bin. Questo caricamento selettivo carica in RAM **esclusivamente il working set necessario**, allineandosi alle strategie dei **motori di inferenza ottimizzati**, e **popola fisiologicamente la page cache del sistema operativo** per gestire i dati su richiesta senza saturare la memoria con l'intero modello.



Apertura del terzo file nei chunk:

Oltre ai due file previsti, deve aprire esplicitamente anche il file **I_{X+1}.bin** del layer successivo. Questo è obbligatorio per **prelevare il trit trasposto** richiesto dalla formula dell'errore (Error_X) della Sezione 5 .
Eccezione strutturale: Durante la retropropagazione dell'ultimo strato (Layer L), questa regola viene interrotta; il file $L + 1$ non esiste e non viene richiesto, poiché l'**errore iniziale viene iniettato direttamente dalla funzione** di perdita senza propagazione a ritroso.

3.1 Prefetching Asincrono e Double Buffering

Per mascherare la latenza fisica dei dischi meccanici (HDD) e prevenire colli di bottiglia durante l'attraversamento dei layer, l'architettura implementa un **sistema di caricamento I/O asincrono** sovrapposto (**Double Buffering**). Il sistema impone un limite hardware rigido che consente la presenza in RAM di un massimo di **due layer simultaneamente** (STREAM_MAX_RESIDENT = 2). Mentre i **thread di calcolo della CPU elaborano** la matematica vettoriale del blocco corrente (X) nel primo slot di RAM, un **thread in background pre-carica i file** I_{X+1} **e** S_{X+1} nel secondo slot inattivo. Questo meccanismo garantisce un **passaggio fluido tra i layer,**

sovrapponendo i tempi di I/O a quelli di calcolo senza violare i vincoli di memoria dell'host.

4. Formati a Virgola Fissa e Accumulatori in RAM

Tutta l'aritmetica di aggiornamento avviene tramite **formati interi**, definendo rigidamente la divisione bit interi/decimali, **supportata da accumulatori temporanei non persistenti** residenti in RAM durante il ciclo a 8 inferenze:

1. **Shadow (**int32_t**):** Formato **Q16.15** (1 bit segno, 16 bit interi, 15 bit decimali), La scelta dei **32 bit garantisce che l'overflow sia quasi impossibile** ma comunque estremamente controllato, **perdendo però il 50% dei valori**. La soglia di isteresi **T** è fissata a $213.$ e opera come discriminante logico; il passo di aggiornamento (Sezione 7) scala questo vincolo, e per un tipico $Update_{Gain} = 4$ equivale a un accumulo minimo di $\Delta = 256$ per scattare.
2. **Output (**int32_t**):** Accumulatore temporaneo in formato **Q16.15** (1 bit segno, 16 bit interi, 15 bit decimali), **inizializzato a 0** per ogni neurone/canale all'inizio del Forward Pass e **aggiornato iterativamente tramite operazioni standard** di moltiplicazione e accumulo (SIMD):

$$Output_{grezzo} \mathrel{+}= (Trit_{ij} \times (Input \gg decay))$$

Al termine del calcolo del layer, il **valore shiftato** viene generato applicando il **fattore di scala globale**:

$$\text{Se } \gamma_{shift} > 0 \Rightarrow Output_{shiftato} = Output_{grezzo} \ll \gamma_{shift}$$
$$\text{Se } \gamma_{shift} < 0 \Rightarrow Output_{shiftato} = Output_{grezzo} \gg |\gamma_{shift}|$$

Al cui **successivamente** viene **applicata la funzione di attivazione** SiLUNS descritta prima per ottenere il **valore definitivo**.

$$Output_{finale} = SiLUNS(Output_{shiftato})$$

3. **Gradiente (**int16_t**):** Formato **Q4.11** (1 bit segno, 4 bit interi, 11 bit decimali).
4. **Learning Rate / LR (**uint16_t**):** Formato **UQ0.16** (0 bit segno, 0 bit interi, 16 bit decimali).

5. **Accumulatore di Errore Assoluto Acc_Err(**uint32_t**): UQ7.25** (0 bit segno, 7 bit interi, 25 bit decimali) Variabile in RAM per sommare iterativamente i valori assoluti degli errori registrati nelle inferenze 1–7.
6. **Accumulatore di Gradiente Acc_Grad(**int64_t**):** Variabile in RAM per sommare iterativamente i gradienti grezzi nello stesso intervallo temporale dell'accumulatore di errore assoluto.
7. **Fattore di Scala Gamma-Shift (γ_{shift}) (int8_t):** Un intero a 8 bit con segno (sufficiente per contenere range di shift hardware standard da -31 a $+31$). Memorizza direttamente l'operando da passare all'istruzione di bit-shift di output al termine del forward pass, occupando solo 1 byte.

4.1 Contratto Numerico HFP (Hybrid Fixed Point)

Nonostante il divieto architetturale di utilizzare moltiplicatori in virgola mobile per il calcolo matriciale dei pesi (sostituiti interamente da addizioni logiche e bit-shift), **l'architettura si definisce Ibrida (HFP)** in quanto **circoscrive l'uso della virgola mobile** (FP32/Double) **esclusivamente alle operazioni di stabilizzazione vettoriale**, esterne al motore di inferenza primario. Nello specifico, il contratto numerico del sistema impone la seguente divisione:

Dominio Intero/Virgola Fissa (Zero FPU): Pesi Ombra **(Q16.15)**, Pesi Ternari **(trit)**, Gradienti **(Q4.11)**, Parametri di Decay **(UQ4.12)** e Scala Globale γ_{shift} **(int8_t)**.

L'intero attraversamento matriciale dei layer (forward e backward pass) avviene **senza l'ausilio di istruzioni floating-point**.

Dominio Virgola Mobile (FP32/Double): L'utilizzo della FPU è **limitato rigorosamente alle normalizzazioni (RMSNorm)**, al mantenimento dello **stato ricorrente temporale** (formato RWKV) e al calcolo terminale della **funzione di perdita** (Softmax/Cross-Entropy). Questa ibridazione spaziale è un compromesso matematico indispensabile: **preserva l'accelerazione O(1)** e il bassissimo consumo di RAM dei pesi ternari per il **95%** dei calcoli, sfruttando la virgola mobile solo nelle funzioni **non-lineari di routing** per prevenire il **collasso numerico del gradiente**.

4.2 Integrazione del Modello di Linguaggio (Architettura RWKV)

A differenza dell'architettura Transformer standard, il motore neurale **QATNN non fa uso del meccanismo di Attention causale multi-testa** (Multi-Head Attention) **né** della costosa **operazione matriciale Softmax spaziale**. Tali operazioni richiederebbero infatti una **complessità computazionale quadratica**, incompatibile con i vincoli hardware di un motore in virgola fissa per CPU. Il modello **implementa invece la ricorrenza lineare RWKV-6 (Finch)**, abbinata a una **struttura Pre-Norm e a un blocco MLP di tipo SwiGLU** ($ff = SiLUNS(W_g \cdot x) \cdot (W_u \cdot x)$).

L'elaborazione del testo avviene tramite un **vocabolario BPE (Byte Pair Encoding)**

integrato a una **matrice di Embedding ternaria**. Sfruttando la **memoria temporale** tipica delle **RNN** (calcolata in tempo lineare e riga per riga), il sistema elimina completamente i colli di bottiglia legati all'Attention, permettendo al motore ternario di **processare documenti complessi a risorse costanti**.

5. Propagazione dell'Error e Calcolo del Gradiente (Layer Lookup)

Nel sistema a blocchi con file separati, la backpropagation **non tiene tutto in RAM** ma procede a ritroso **caricando i file del gruppo corrente** X. Sfruttando i dati salvati nel forward pass, il **calcolo avviene tramite le seguenti relazioni algebriche**:

1. **Inizializzazione dell'Errore sull'Ultimo Layer (**Output**):**
   La retropropagazione parte dall'ultimo layer della rete (layer L). L'errore iniziale viene calcolato come **differenza** diretta tra l'**output predetto** dal modello e il **target (valore atteso) nel formato intero di riferimento:**

$$Error_L = Output_{predetto} - Target_{atteso}$$

   Funzione di perdita:

$$Loss = \Sigma_{k=1\ldots N}(Output_k - Target_k)^2 \gg 19$$

   Nota architetturale: Il $\gg 19$ quantizza **esclusivamente** il valore riportato della perdita per la stampa, non la funzione differenziata. La derivata che entra nel calcolo del gradiente G_L è semplicemente: $\frac{\partial Loss}{\partial Output_L} = 2(Output_L - Target_L)$ in **Q16.15** (senza shift).
   L'unico shift a 19 bit applicato per l'**ultimo strato** è quello del **prodotto esterno** descritto al punto 3, indispensabile per convertire l'accumulo da **Q32.30** a **Q4.11**.

2. **Calcolo dell'Errore per il Gruppo Corrente (X):**
   Per propagare l'errore verso l'inizio della rete, si moltiplica l'errore ricevuto dal gruppo successivo ($X + 1$) per i **Trit Trasposti del gruppo successivo (**$X + 1$**)**:

$$Error_X = Trit^T_{X+1} \times Error_{X+1}$$
$$(Risultato\colon\ un\ vettore)$$

3. **Calcolo del Gradiente per i Pesi del Gruppo Corrente (X):**
   Per determinare la correzione dei pesi del blocco attuale, si prende l'**errore del gruppo successivo** ricevuto e lo si moltiplica per l'**Input del gruppo corrente (**X**)** (coincidente con l'output salvato del gruppo attuale X):

$$G_X = (Error_{X+1} \otimes Input_X) \gg 19$$

$$(Risultato\ logico\colon\ prodotto\ esterno\ in\ formato\ Q4.11\ )$$

Per **impedire che gli errori di picco causino un overflow** distruttivo durante la propagazione vettoriale, il risultato **viene forzato preventivamente** (clipping) all'interno del campo di **saturazione nativo del formato Q4.11**, ovvero l'intervallo rigoroso $[-2047, +2047]$.

Nota architetturale sull'allocazione (Calcolo on-the-fly): Poiché il gradiente per un singolo token deriva da un prodotto esterno di rango 1, la **matrice G_X non viene mai materializzata né allocata interamente in RAM**. L'elaborazione procede rigorosamente riga per riga: il gradiente viene calcolato, applicato istantaneamente per l'aggiornamento dello Shadow Weight e subito scartato. Questa ottimizzazione **abbatte il requisito di memoria per il gradiente da 26 GB a frazioni di Megabyte: 26 GB è 2 byte per parametro, dunque vale solo per modelli con oltre 13 miliardi di parametri. Su un modello da 7 miliardi la stima richiesta è 14 GB, non 26**, garantendo la fattibilità della backpropagation a blocchi.

Nota di allineamento formati: Poiché il prodotto esterno tra l'errore e l'input (entrambi in formato **Q16.15**) genera nativamente un accumulo intero in **Q32.30**, viene applicato uno **shift a destra di 19 bit (**$\gg 19$**)** per convertire rigorosamente il dato nel formato **Q4.11** del gradiente. Questo troncamento algebrico **impedisce la saturazione** distruttiva dei registri e previene l'autospegnimento dell'ottimizzatore.

Vincolo di Sicurezza e Accumulo: L'accumulo del prodotto esterno tra errore e input viene eseguito temporaneamente in un **registro a 64 bit** (`int64_t`) per **preservare l'intera precisione del formato Q32.30** prima dell'applicazione dello shift a destra di 19 bit. Questo approccio elimina qualsiasi restrizione artificiale sul campo d'ingresso, garantendo la compatibilità con l'intero intervallo rappresentabile dal formato **Q16.15** senza saturazioni distruttive.

4. **Accumulatore di Errore Assoluto Acc_Err(**uint32_t**):** Variabile di supporto **non persistente su file** (residente in RAM) utilizzata per **sommare i valori assoluti degli errori durante le 7 inferenze** preliminari, azzerata al 8° ciclo dopo il calcolo del Learning Rate.

6. Ottimizzazione del Decay (UQ4) e Gestione dei Pesi di Supporto

Il parametro di decadimento (decay) che governa il bit-shift non è statico, ma viene ottimizzato iterativamente durante la backpropagation tramite un valore di supporto ad alta precisione.

- **Shadow del Decay:** Memorizzato come un intero a 16 bit senza segno in formato **UQ4.12** (4 bit interi, 12 bit frazionari).

  Inizializzazione:

  $$decay_{Weight} = min(15, round(0.5 \times log_2(N_{in}))) \times 2^{12}$$

  troncato al campo **UQ4.12**. Il valore iniziale è quello che riporta l'accumulatore $Output_{grezzo}$ vicino a 1: partire da 0 moltiplica l'accumulatore per circa 1000 e manda γ_{shift} fuori campo.

  N_{in} è la lunghezza dell'**asse di riduzione del prodotto scalare** nel passaggio in avanti (forward pass), per canale di output: indica **quanti input convergono su ciascun output**.

- **G_decay** = Arriva calcolando l'**errore rispetto al parametro di decadimento**. In pratica, durante il passaggio a ritroso (**backprop**), misuri quanto l'errore corrente risente del valore di shift attuale:

  $$G_{decay} = ((\Sigma |Error_n \times Input_n|) \gg 8) \,\&\, 0xFFFF$$

  (L'accumulo viene shiftato e mascherato a 16 bit per prevenire overflow prima dell'aggiornamento del parametro)

  Questo gradiente indica la **direzione e l'intensità con cui bisogna correggere il bit-shift** (aumentandolo o diminuendolo) per migliorare l'attenuazione del segnale.

- **Processo di Aggiornamento e Quantizzazione (Cut):**
  1. **Delta_decay** = (G_decay * LR) >> 15
  2. **decay_Weight** = decay_Weight - Delta_decay
  3. **Estrazione (Cut):** Per l'inferenza successiva, la parte frazionaria viene troncata per estrarre l'intero puro destinato all'operazione di bit-shift: decay = decay_Weight >> 12 *(Il valore finale viene sottoposto a clipping in un range sicuro, es. 0-15).*

7. Backpropagation e Aggiornamento dei Pesi

Il calcolo della correzione sfrutta i bit-shift per evitare overflow e allineare i formati, scartando i bit decimali in eccesso.

1. $decay_{Weight} = \min(15, (31 - _builtin_clz(N_{in})) \gg 1) \ll 12$

   ***Nota architetturale: Update_Gain** è un intero con **0 <= Update_Gain <= 12**, per conciliare **i limiti del formato UQ0.16** del **Learning Rate** (saturazione a 65535) con l'alta soglia di isteresi (2^{13}). Riducendo lo scorrimento a destra, **permette ai gradienti reali di accumulare magnitudo sufficiente a sbloccare lo stato del trit** senza richiedere l'uso di formati dati a 32 bit per il LR.*
2. **Shadow** -= Delta_Shadow

7.1 Meccanismi Strutturali di Convergenza e Stabilità

Affinché il modello ternario superi la fase di stallo iniziale e i gradienti non **subiscano saturazioni** distruttive, l'architettura **implementa quattro leve di convergenza** essenziali, le quali operano in sinergia durante l'aggiornamento dei pesi:

Ridimensionamento del Gradiente (Grad-Scale): Il prodotto esterno grezzo, **calcolato in Q32.30** e shiftato **a destra di 19 bit**, genera un magnitudo **intrinsecamente troppo elevato** per il **formato di destinazione Q4.11**. Il sistema applica pertanto un **meccanismo di attenuazione** basato su una **media mobile a 8 passi** combinato a un blocco rigido (**clamping**), mantenendo gli aggiornamenti entro un **range di tolleranza sicuro** e prevenendo l'esplosione dei gradienti.

Calibrazione del Guadagno (Update_Gain): Nel calcolo del Delta per i Pesi Ombra ($\Delta_{Shadow} = (G \times LR) \gg (12 - Update_Gain)$), il parametro $Update_Gain$ è **fissato di default a 0**. Questa precisa configurazione matematica permette all'equazione di **accumulare l'inerzia esatta** necessaria per **superare la soglia di isteresi** ($T = 213$), eludendo al contempo la saturazione strutturale del **Learning Rate nel formato UQ0.16**.

Rumore Gaussiano Iniziale (Sigma): All'avvio, i Pesi Ombra **non vengono inizializzati prossimi allo zero**. Vengono invece **forzati attraverso una distribuzione gaussiana con deviazione standard** $\sigma = 2048$ (circa 10 volte la soglia di isteresi). Questo rumore asimmetrico estremo è vitale: **rompe la simmetria del modello** e assegna una direzione iniziale ai pesi **prima ancora che la rete processi il primo gradiente**.

Warmup e Stall-Halving (WSD): L'addestramento sfrutta una curva WSD **(Warmup-Stable-Decay)**. I primi 10 passi di calcolo **fungono da riscaldamento per stabilizzare i gradienti derivati**, implementando **parallelamente** un **dimezzamento dinamico del passo di apprendimento** (Stall-Halving) qualora la funzione di perdita registri una situazione di stallo prolungato.

8. Ciclo Adattivo a 8 Inferenze (%X e LR)

Il sistema valuta e aggiorna lo stato di apprendimento a blocchi di 8 inferenze per garantire stabilità, evitando completamente divisioni in favore dei bit-shift:

- **Fase A (Inferenze 1 - 7):** Raccolta incrementale e accumulo in parallelo dei valori assoluti degli errori e dei gradienti grezzi negli accumulatori in RAM.
- **Fase B (Inferenza 8)**:
  - **Approssimazione del Gradiente Medio (G_{avg}):** Poiché dividere per 8 esatta richiede divisioni costose, si utilizza l'approssimazione bitwise per divisione per 8 o 16 (es. approssimando la finestra a 8 o 16 step, oppure moltiplicando per il reciproco fisso in virgola fissa). Usando una finestra di 8 inferenze, il calcolo diventa un semplice shift a destra di 3 bit:
    $$G_{Avg} = (Acc_{Grad} + G_{Curr}) \gg 3$$
  - **Noise Clipping Preventivo:** Il Clipping Preventivo non opera su Acc_Err: è un **accumulatore unsigned che satura per costruzione** e il suo campo è già limitato a **UQ7.25**. L'unica grandezza che richiede clippaggio è il gradiente grezzo G, ed è **già vincolata a [-2047, +2047]** dalla sezione 5.

    Calcolo del Learning Rate (LR) e Raffreddamento Esplicito:
    Per **evitare** che l'**ottimizzatore converga** prematuramente su un **punto fisso** (bloccando l'apprendimento), l'aggiornamento rimuove l'**inerzia autoreferenziale** e adotta un meccanismo di **decadimento esplicito guidato** esclusivamente dall'errore residuo. Il calcolo applica uno *schedule* di raffreddamento (Simulated Annealing) incrementando dinamicamente il divisore tramite bit-shift:

    $$LR_{new} = max(64, min(65535, (|G_{Avg}|) + (Acc_{Err} \gg (9 + Cooling_{Shift}))))$$

    L'utilizzo esplicito delle parentesi all'esponente **(9 + Cooling_Shift)** garantisce in **fase di implementazione** che il raffreddamento **agisca rigorosamente come un divisore crescente**, comprimendo il passo di apprendimento invece di sommarsi ad esso.

    Decadimento Asintotico (Cooling_Shift**):** È un parametro intero (inizializzato a 0) che si incrementa gradualmente (es. ogni 1024 macro-epoche). Con l'**avanzare dell'addestramento**, la divisione passa da $\gg 9$ a $\gg 10, \gg 11$, comprimendo attivamente il passo di apprendimento man mano che la rete si avvicina al minimo globale.
    Aggancio al Pavimento (64): L'operatore **max** impone il limite strutturale nel formato **UQ0.16**. Quando l'errore crolla o il **raffreddamento diventa estremo**, il parametro si poggia sul valore di 64, garantendo uno spazio per i

micro-aggiustamenti (fine-tuning) terminali senza mai causare lo spegnimento per collasso matematico a zero.

9. Gestione Adattiva della Profondità (Layer Cut)

Il calcolo del numero di file layer da caricare dalla percentuale **%X** utilizza l'aritmetica intera nativa del linguaggio C:

$$\min\left(totlayer, \frac{\min\left(\max\left(40, totlayer\right), 42949672\right) \times p}{100}\right) \text{-> uint32_t}$$

- **Ruolo del parametro 40:** Il valore **40** agisce come operando di **soglia minima** nel fattore di **moltiplicazione per percentuali ridotte su modelli compatti**, evitando che il **numero di file scenda a zero prematuramente**. Nelle reti con meno di 40 layer, il **min** esterno restituisce correttamente il totale effettivo (**totlayer**)
- **Troncamento intero:** La divisione intera della CPU tronca nativamente la parte decimale senza richiedere librerie matematiche. **La perdita di precisione è confinata al troncamento algebrico della percentuale**, senza impattare la stabilità del caricamento selettivo.

10. Sitografia

1. BitNet & BitLinear (Pesi Ternari)

- **The Era of 1-bit LLMs (BitNet b1.58):** arXiv:2402.17764
- **BitNet b1.58 2B4T Technical Report:** arXiv:2504.12285
- **Repository Ufficiale Microsoft BitNet:** GitHub - Microsoft/BitNet

2. Architettura RWKV

- **RWKV: Reinventing RNNs for the Transformer Era:** arXiv:2305.13048
- **Repository Ufficiale RWKV-LM:** GitHub - BlinkDL/RWKV-LM

3. Quantizzazione e Formati Fixed-Point (Q-Format)

- **Quantizing the RWKV Family (PTQ):** OpenReview - RWKVQuant
- **Fixed-Point Arithmetic & Scaling in Neural Networks:** KIT Publications - Fixed-Point Quantization
- **Jacob, Kligys, Chen et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference"**, CVPR 2018 (arXiv:1712.05877). Il presente modello si distingue da questa base teorica sostituendo l'inferenza intera pura con un regime ternario governato da pesi ombra (Shadow Weights) e mappa di isteresi dinamica.

4. Letteratura Avanzata sul Ternario (2026)

- **Sparse-BitNet**: *Sparse-BitNet: Scaling 1-bit LLMs with Structured Sparsity*, arXiv:2603.05168 (2026).
- **TernaryLM**: *TernaryLM: Native Ternary Large Language Models*, arXiv:2602.07374 (2026).

5. Vocabolario BPE a caratteri utilizzato dal motore per l'elaborazione del testo

- **"Neural Machine Translation of Rare Words with Subword Units" (2015):** arXiv:1508.07909 Sennrich, R., Haddow, B., & Birch, A.

6. Riferimento per la funzione di attivazione SiLU originaria

- **"Sigmoid-weighted linear units for neural network function approximation in reinforcement learning" (2017):** arXiv:1702.03118 Elfwing, S., Uchibe, E., & Doya, K.

7. Applicazione dei paradigmi standard di sovrapposizione I/O (Asynchronous I/O Overlap) derivati dalle architetture di calcolo ad alte prestazioni (HPC), adattati per vincoli di memoria MAX_RESIDENT.

Calculations Powered By:

Gemini 3.1 Pro-Extended
Space Bunny 1.0 [Freebuff]
Muse Spark 1.3 [Freebuff]
My Own Project ideas and Inputs